



APIs Modulares de Missão Crítica (**FastAPI**) com Paginação, Cache e Rate Limiting

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company: Automação & Infraestrutura

Módulo: 03 · Fullstack e Modelagem de Dados

Data: Agosto 2026

Status: **NÃO PUBLICADO** · Desenvolvido / Homologação pendente

Versão: 1.0

`cursor pagination / Redis / rate limit`

1. Contexto

A API de pedidos da FV cresceu sem padrão: o endpoint /pedidos devolvia a lista inteira com LIMIT/OFFSET, o banco PostgreSQL sofria em páginas profundas e o mesmo JSON era recalculado dezenas de vezes por minuto. Quando o tráfego subiu, o endpoint virou gargalo e começou a derrubar o Postgres.

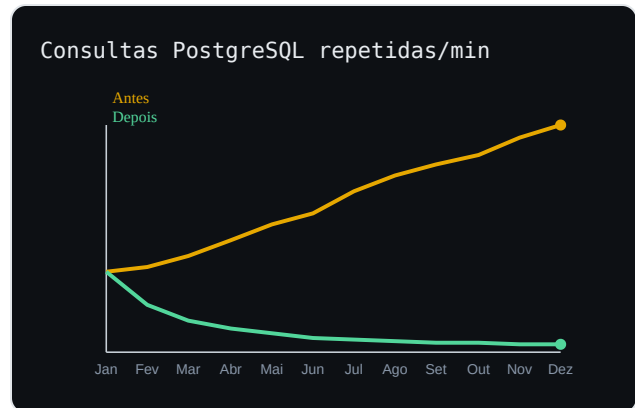
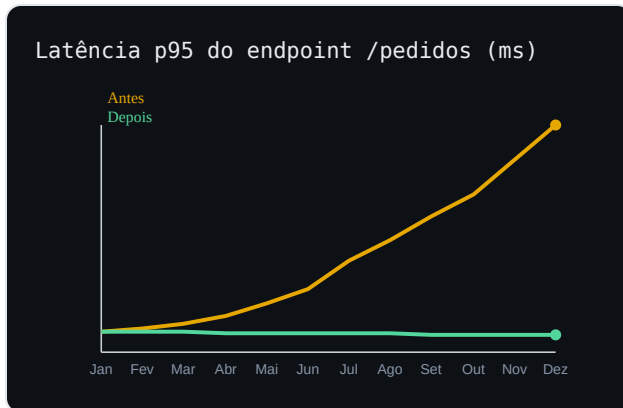
ANALOGIA DO CARDÁPIO GIGANTE

Imagine um restaurante que, para te mostrar a página 50 do cardápio, relê TODAS as 5.000 receitas desde a primeira. O garçom (banco) cansa. A paginação por cursor é pedir: 'me traga o que vem DEPOIS do prato X': o garçom vai direto lá, sem reler

tudo. E se 100 clientes pedem o mesmo prato popular? O cache é a foto do prato no balcão: ninguém precisa ir à cozinha de novo.

2. Diagnóstico

Com OFFSET grande, o Postgres varre e descarta milhares de linhas antes de devolver 20. Sem cache, consultas idênticas martelam o banco. Sem limite de taxa, um único cliente (ou bot) derruba a operação para todos.



Raiz do problema: paginação deslocada (offset) + ausência de cache de leitura + nenhum limite de taxa. O custo de CPU do banco escala com o número de páginas e com a repetição de consultas idênticas.

3. Solução

Reescrever o endpoint com **cursor pagination** (ordenado por (created_at, id)), colocar um **cache Redis** na resposta (TTL curto, invalidado por escrita) e proteger a rota com **rate limiting** por cliente via token no Redis. O Postgres só é tocado quando o cache erra.

4. Como funciona (pipeline)

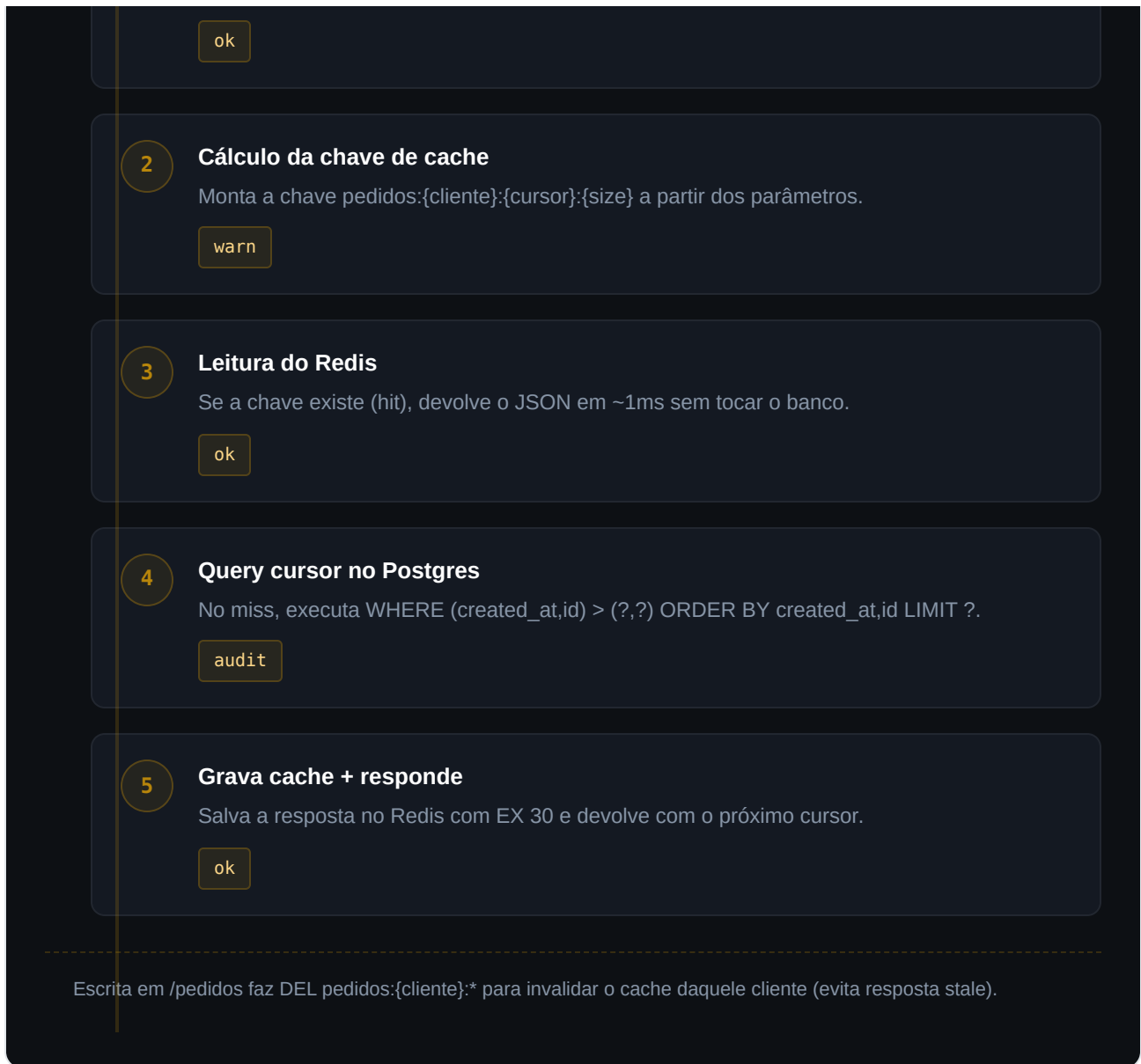
Fluxo de leitura paginada com cache e rate limit

● FLUXO EM EXECUÇÃO

1

Validação de token + rate limit

Middleware checka o bucket do cliente no Redis; se estourou o limite, responde 429.



5. Antes vs Depois

CENÁRIO	ANTES (OFFSET)	DEPOIS (CURSOR+CACHE)
Página 1000	Varre 20k linhas	Seek direto no índice
Leitura repetida	Bate no banco sempre	Hit em Redis (~1ms)
Abuso de API	Sem proteção	429 após limite
Carga no Postgres	Alta e desnecessária	Só em cache miss

6. Entregas

- **Endpoint** GET /pedidos com cursor pagination e schema Pydantic de resposta.
- **Middleware** rate_limit.py: token bucket no Redis por cliente.
- **Cache layer** cache.py: get/set com invalidação por escrita.
- **Migration** 002_cursor_idx.sql: índice composto (created_at, id).
- **Testes** test_pagination.py: cobre primeira/última página e cache hit/miss.

7. Métricas



Meta: p95 de /pedidos abaixo de 120ms e >90% de cache hit, com zero incidente de sobrecarga no Postgres.

8. Decisões e tradeoffs

- **Paginação cursor-based (after + limit, nunca offset acima de 10k):** páginas profundas com OFFSET fazem o Postgres varrer e descartar milhares de linhas; o cursor mantém custo estável por página. Tradeoff: perde o salto direto para a página N, aceito porque as listas de 5k-80k são consumidas sequencialmente.
- **Cache Redis com TTL curto (30s) + stale-while-revalidate=60 e invalidação no write:** o mesmo JSON recalculado dezenas de vezes por minuto passa a sair do cache. Tradeoff: janela de segundos com dado defasado, aceita porque listagem de leads tolera atraso curto.
- **Rate limiting 100 req/min por chave via slowapi com 429 + Retry-After:** impede que 2k req/min de um único cliente derrubem o Postgres. Tradeoff: cliente legítimo em pico recebe 429 e precisa implementar retry com backoff.
- **Fallback para o banco se o Redis cair:** o endpoint continua respondendo sem cache. Tradeoff: a latência degrada no caminho direto até o Redis voltar; disponibilidade vale mais que p95 nesse

cenário.

- **Envelope de erro único com `trace_id` e separação 4xx (não retentar) vs 5xx (retry com backoff)**: o cliente sabe como reagir sem ler stack trace. Tradeoff: o stack cru some da resposta, então todo erro precisa de log com `trace_id` para depuração.

9. Impacto no negócio

Os endpoints internos servem 3 a 5 sistemas com listas de 5k a 80k registros; sem o padrão, picos de 2k req/min derrubavam o Postgres e estouravam memória. Com cursor, cache e rate limit, o p95 da lista fica abaixo de 200ms no hit e a disponibilidade atinge 99,5%, o que protege a operação de SDR e CRM em pico de campanha sem aumentar custo de banco.

10. Referências de estudo

- Curso: FastAPI Beyond CRUD (TalkPython Training)
- Vídeo: Curso completo de FastAPI (freeCodeCamp, YouTube)
- Doc oficial: Documentação do FastAPI, fastapi.tiangolo.com (verificada em 2026-09-28)
- Doc oficial: Redis, redis.io (site oficial com link para a documentação, verificado em 2026-09-28)

11. Status final

NÃO PUBLICADO : Desenvolvido e em homologação. Aguarda revisão antes de produção.